value received by the second layer output neuron, and this offsets the positive signal received from neurons 0 and 4. However, when the input is 1s only on x_3 and x_{11}, that negative signal is greatly diminished by the fact that row 3 of the Layer 1 weight matrix has opposite signs in columns 3 and 11. This explains why the network has a strong learned preference for the inputs that appear in the same clause to have opposite Layer 1 weights for negative rows.

With this as motivation, let us get back to the separation of the weight matrix into the rows that have positive Layer 2 weights and those that have negative Layer 2 weights (bottom of Figure 2). In the network, the weights of the Layer 2 neuron W_2 defines whether a row of the W_1 matrix contributes to the final result as a positive or a negative value. Essentially, the positive W_2 weights correspond to features in Layer 1 that we call *positive witnesses*: they provide evidence that the input is a 1 (True) instance and provide a strong positive signal to support that. Negative W_2 weights are *negative witnesses* and provide a negative signal due to evidence that the input is a 0 (False) instance. The lower left image in Figure 2 shows that the negative witnesses are essentially computing an XOR: if one variable of a pair of variables is present but not the other to offset it, that provides a signal that this is a No instance.

As we discuss in the next section, feature channel coding allows the neural network to compute in "superposition" [1], and in a different example than the one presented in this section, the number of features encoded can be much larger than the number of neurons.

3.3 Combinatorial Interpretation Using Feature Channel Coding

We next point out that our techniques go beyond just understanding the structure of the underlying network, and in fact can, at least in some cases, be used to interpret the computation explicitly. From Figure 2, it seems likely that we can determine the underlying randomly chosen Boolean formula from only the learned parameters of the network. Here we address the question of whether we can do this for formulas chosen from the distribution described for Figure 2, although we found the technique we describe to be effective across a broad range of formula distributions. We demonstrate that even just the Layer 1 weight matrices are sufficient to reconstruct the entire formula, as long as the trained network was provided with enough training samples to generalize fairly well. In fact, we found that a straightforward algorithm allows us to reconstruct the formula (for this 2-variable AND special case): we take the absolute values of all entries of the Layer 1 weight matrix, and then find for each column of the resulting matrix which other column it is most closely correlated with. If column i is most closely correlated with column j, then we assume that input i is paired with input j in the Boolean formula. Note that this algorithm does not necessarily return symmetric results.

The results of our tests on this algorithm are depicted in Figure 3. We tested a range of sizes of the hidden layer (and corresponding size of Boolean formula), as well as a range of sizes of the training set. For each of these, we chose ten random formulas, and then trained each formula on a set of random inputs. We then performed two tests: we measured the error rate of the trained network on a new test set of inputs, chosen from the same distribution as the training set, and we tested the accuracy of the correlation of absolute values algorithm. Our measure of success for the correlation algorithm was how many of the pairings it was able to correctly identify. We see that for sufficiently large sets of training data, our algorithm was almost always able to reconstruct the entire formula correctly. Furthermore, the measure of "sufficiently large" here seems to tie closely to the network correctly learning the function (as measured by the test set): when there was enough training data for the error rate on the test set to be small, the correlation algorithm performed

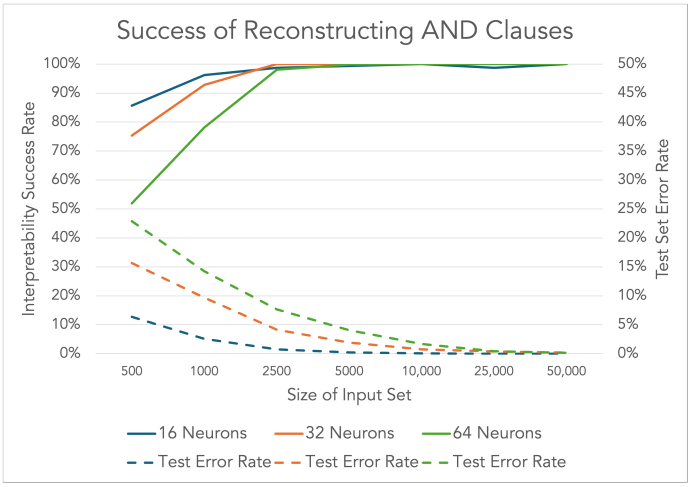

Figure 3: Results on our algorithm to recreate a Boolean formula from its Layer 1 weight matrix.

nearly perfectly.

Putting all this together, the gradient descent training process of the network has learned to perform ANDs of the input channels in the first layer, and an OR of the AND clauses coming out of the first layer in the second layer.

3.4 Polysemanticity and Superposition

Let us touch a bit more on the notions of polysemanticity and superposition [1, 12, 17, 19]. When one looks at the set of neurons of a feature channel code as in our example above, we see that there is overlap. The same neuron is polysemantic and fires in response to multiple features. The accepted idea for how superposition is implemented is that neurons represent features using polysemanticity. To quote [12]: "in the superposition hypothesis, features are represented as almost-orthogonal directions in the vector space of neuron outputs. Since the features are only almost-orthogonal, one feature activating looks like other features slightly activating." This is true, and yet our theory of feature channel coding suggests that superposition can also be explained by the combinatorial property of using feature channel codes that overlap. The noise of one feature slightly lighting up others is explained not just by alignment in vector space, but also by the use of multiple rows in the feature channel coding so the relative importance of a given neuron is lowered [1]. Moreover, we don't need to decipher directionality in activation space or geometric representations in order to map the features captured by a collection of neurons: the feature channel coding in the weight matrices allows us to do it combinatorially without any activation information. Furthermore, as we will see in Section 4, using the combinatorial interpretation also allows us to analyze the code pattern distributions of networks to explain scaling laws in ways that seem impossible to capture by looking at features through their directionality in activation space.

Getting back to our example network, if overlapping codes introduce noise, given that our network was trained on 8 clauses using 16 neurons in the first layer, the training could have found a coding that had one code row per clause: one monosemantic neuron per feature. In this way, there would be no overlaps among codes, and thus no cross-feature noise. However, as in the matrices

resulting from our training, we see that the training settled on multiple overlapping codes and the neurons are polysemantic. In other words, polysemanticity appears even when there are fewer features than neurons.

We believe that there is a combinatorial explanation for this. Gradient descent searches the state space for local minima. There are many more encodings of the desired computation using overlapping codes than using non-overlapping monosemantic ones, and thus likely many more local minima involving feature channel coding using multiple overlapping polysemantic neurons. Thus, gradient descent is more likely to find such an encoding than the monosemantic one, and then not be able to move from that local minimum. This is somewhat reminiscent to the findings of [22]. As witnessed in our example, coding (and with it polysemanticity) does appear in cases where one does not need to compute in superposition. It would be interesting to obtain more evidence or a proof of this hypothesis, which could be done using an approach similar to the one we take in Section 4.6.

We will provide other examples of feature channel coding in the context of other training problems in Sections 4.7 and 5. For example, the interested reader can skip to take a look at an example of a one dimensional"vision problem" in Section 5.2. Before we do so, we provide a quantitative analysis of feature channel coding. This will provide numerical evidence of feature channel coding and also allow us to better understand the constraints imposed by this technique. These constraints in turn will allow us to provide a combinatorial explanation of a scaling law in neural networks.

4 Combinatorial Analysis of a Scaling Law

In this section we use our combinatorial interpretability framework to analyze scaling laws around both *how* a model computes Boolean formulas, as well as *why* the model breaks down as a result of increasing complexity. At the core of our analysis is a study of the strong correlation between a model's accuracy and its ability to encode features. This analysis serves a number of purposes:

- It provides quantitative and convincing evidence that feature channel coding is being used by networks trained via gradient descent to evaluate Boolean formulas.
- It provides insights into the constraints faced by such systems, and why and when they break down.
- It provides an example of the power of the combinatorial approach to analyze such systems.

For this analysis, we will focus on DNF formulas consisting entirely of 4-variable clauses. We will consider two different cases of such formulas: one where all variables appear in their positive form (this section), and one where each clause has 3 positive variables and 1 negated variable (Section 4.7). One of our primary goals is to see how the system reacts to increasing complexity of the formula being learned. To study this, we start with a formula with a small number of clauses, and gradually increase the number of clauses in the formula, until the point the model is no longer able to learn the formula accurately. Note that this does not change the size or structure of the network or the size of its inputs in any way. It merely creates a Boolean formula with a more complex underlying truth table for the model to learn.

In more detail, in all experiments, the network consists of 32 input variables and a single output neuron, computed via a sigmoid. In between is a single hidden layer, and we test $j = 16, 32$, and 64 neurons in that layer. Each hidden layer neuron has a bias followed by a ReLU, and we use an output layer bias as well, and the hidden layer is fully connected to the inputs, and (post ReLU) is